GitHub Copilot — The Complete Developer's Guide

> A comprehensive, structured handbook for developers who want to master GitHub Copilot — from basic autocomplete to advanced agent workflows.

Table of Contents

1. Introduction to GitHub Copilot
2. Inline Suggestions (Autocomplete)
3. Fill In The Middle (FIM)
4. Next Edit Suggestions
5. GitHub Copilot Chat
    - Chat Modes: Ask, Plan & Agent
    - Slash Commands
    - Chat Participants
    - Running Terminal Commands from Chat
    - Exporting Chat Sessions
6. Context Window & Token Management
    - What Is a Context Window?
    - Context Pollution
    - Best Practices for Context Management
    - Viewing Context Usage in VS Code
7. Agent Instructions & Configuration Files
    - copilot-instructions.md
    - AGENTS.md
    - .github/agents/*.md
    - .github/instructions/*.instructions.md
    - Comparison Table
8. Custom Agents
9. Custom Prompts (Reusable Slash Commands)
10. MCP — Model Context Protocol
11. Agent Skills
12. SubAgents
13. Hooks
14. AI-First Project Structure
15. Best Practices & Tips
16. Quick Reference Cheat Sheet

---

1. Introduction to GitHub Copilot

GitHub Copilot is an **AI-powered coding assistant** developed by GitHub (backed by OpenAI models). It integrates directly into your code editor (VS Code, JetBrains, Neovim, etc.) and helps developers write code faster by:

- **Suggesting code** as you type (inline autocomplete)
- **Answering questions** about your codebase via Copilot Chat
- **Generating entire files**, tests, and documentation
- **Executing multi-step workflows** using Agent Mode
- **Integrating with external tools** via MCP (Model Context Protocol)

Key Concept: Non-Deterministic Output

GitHub Copilot is **non-deterministic**. This means:

> If two developers type the **exact same prompt**, Copilot may generate **slightly different code** each time.

This is normal behavior for AI models. The output varies because the model uses probabilistic sampling to generate responses. Don't be alarmed if the suggestions you see differ from what a colleague or tutorial shows.

---

2. Inline Suggestions (Autocomplete)

What Are Inline Suggestions?

Inline suggestions are Copilot's core feature. As you type code, Copilot **predicts what you'll write next** and displays it as **ghost text** (grey text) directly in your editor.

How It Works

1. You start typing code
2. Copilot analyzes the current file, open tabs, and imported modules
3. It displays a grey text suggestion inline
4. You decide to accept, reject, or partially accept it

Accepting Suggestions

Action	Shortcut
Accept full suggestion	`Tab`
Accept word-by-word	`Cmd+→` (Mac) / `Ctrl+→` (Windows)
Dismiss suggestion	`Esc`
See next suggestion	`Alt+]`
See previous suggestion	`Alt+[`

Example: HTML Autocomplete

You type:

```
<!DOCTYPE html>
```

Copilot suggests:

```
<html>
  <head>
    <title>Sample Page</title>
  </head>
  <body></body>
</html>
```

Press Tab to accept the entire suggestion.

Example: JavaScript with Comments

Comments are one of the most powerful ways to **guide** Copilot. When you write a descriptive comment, Copilot uses it as context to generate more accurate code.

```
// query all HTML label tags
```

Copilot suggests:

```
const labels = document.getElementsByTagName("label");
```

Key Insight: Comments give Copilot **context**. The more descriptive your comments, the better the suggestions. Think of comments as lightweight prompts.

Visual Indicators in VS Code

Color	Meaning
Grey ghost text	Inline suggestion (autocomplete)
Blue highlight	Suggested new code to add
Red highlight	Code that Copilot suggests removing

---

3. Fill In The Middle (FIM)

What Is FIM?

Fill In The Middle is a Copilot feature that **completes missing code between two existing code blocks**. Instead of only predicting what comes next, Copilot looks at both the code **before** and **after** the cursor to fill the gap intelligently.

How It Works

```
Code before cursor  →  Copilot analyzes both sides
          +
Code after cursor   →  Predicts the best code for the gap
```

Example

You have this existing HTML structure:

```
<head> </head>
```

When you place your cursor inside the `<head>` tags and press Enter, Copilot may suggest:

```
<meta charset="UTF-8" /> <title>Page</title>
```

When FIM Is Useful

- Filling in function bodies when the signature and return type exist
- Adding middleware between existing route definitions
- Completing configuration objects
- Writing code between existing blocks of logic

---

4. Next Edit Suggestions

What Are Next Edit Suggestions?

Next Edit Suggestions go **beyond simple autocomplete**. Instead of just predicting what you'll type next, Copilot scans your **entire file** and suggests **changes, improvements, or refactors** to existing code.

How It Works

Copilot watches your editing patterns and suggests logical next edits. For example, if you create a new function that duplicates logic already existing elsewhere in the file, Copilot will suggest refactoring the old code to use the new function.

Example: Automatic Refactoring

Step 1: You have existing code:

```
window.onload = function () {
  const labels = document.getElementsByTagName("label");
};
```

Step 2: You create a new function:

```
function getAllLabels() {}
```

Step 3: Copilot suggests filling the new function:

```
function getAllLabels() {
  return document.getElementsByTagName("label");
}
```

Step 4: Copilot then suggests updating the old code:

```
window.onload = function () {
  const labels = getAllLabels();
};
```

This automatic refactoring saves time and keeps code DRY (Don't Repeat Yourself).

Quick Comparison of Code Suggestion Features

Feature	What It Does
Inline Suggestions	Predicts the next code while you're typing
Fill In The Middle	Completes missing code between existing lines
Next Edit Suggestions	Suggests improvements or refactors to existing code

---

5. GitHub Copilot Chat

Copilot Chat is an **interactive AI assistant** embedded in VS Code's sidebar (or inline). You can ask questions, request code generation, debug errors, and execute complex multi-step tasks — all through natural language conversation.

5.1 Chat Modes: Ask, Plan & Agent

Copilot Chat has **three modes** that you can switch between freely within the same conversation:

Mode	Purpose	Modifies Code?
Ask Mode	Q&A — ask questions, get explanations	No
Plan Mode	Creates a structured implementation plan	No
Agent Mode	Executes code changes, runs commands, uses tools	**Yes**

Ask Mode

Use Ask Mode when you want to:

- Understand how a piece of code works
- Get explanations of concepts or errors
- Ask about best practices
- Explore options before committing to a change

Copilot generates responses but does **not** modify your project files.

Plan Mode

Use Plan Mode when you want to:

- Outline an approach before implementing
- Break a complex task into steps
- Get a structured plan you can review and refine

The plan can be revised interactively — you can ask Copilot to adjust steps before executing anything.

Agent Mode

Agent Mode is where the real power lies. In this mode, Copilot can:

- **Read and write files** in your project
- **Run terminal commands** (npm install, tests, etc.)
- **Use MCP tools** (database queries, API calls, etc.)
- **Execute multi-step workflows** autonomously

Recommended Workflow

```
1. Start in Plan Mode    →  Outline the approach
2. Review and refine     →  Adjust the plan interactively
3. Switch to Agent Mode  →  Execute the plan
4. Switch to Ask Mode    →  For questions mid-conversation
```

Applying Code from Ask/Plan Mode

When Copilot generates code snippets in Ask or Plan mode, you don't need to copy-paste. You can:

- Click the **Apply** icon to insert the snippet directly into the correct file
- Click the **Terminal** icon to insert a command into the terminal

---

5.2 Slash Commands

Slash commands are **prebuilt prompt templates** that trigger common tasks. Instead of writing detailed prompts, you type `/command` and Copilot executes the built-in template.

Key Slash Commands

Command	Purpose
`/setup-tests`	Configure a testing framework for your project
`/tests`	Generate unit tests for selected code
`/explain`	Explain how selected code works
`/fix`	Suggest a fix for a problem
`/doc`	Generate documentation for selected code
`/optimize`	Suggest performance improvements

Example: `/setup-tests`

When you run `/setup-tests`, Copilot may:

1. Initialize an npm project (`npm init`)
2. Install testing libraries (`npm install --save-dev jest`)
3. Create configuration files (`jest.config.js`)
4. Generate an initial test file (`script.test.js`)

Example: `/tests`

1. Select a function in your editor
2. Type `/tests` in the chat
3. Copilot generates comprehensive test cases including:
    - Standard test cases
    - Edge case tests
    - Validation tests

5.3 Chat Participants

Chat participants let you **target specific areas** of VS Code when prompting Copilot:

Participant	Purpose	Example
`@workspace`	Work with your entire project	`@workspace /explain`
`@terminal`	Work with terminal output	`@terminal explain the last error`
`@editor`	Work with the currently open file	`@editor what does this function do?`

Combining participants with slash commands creates precise, targeted prompts:

```
@workspace /explain    →  Explain the project structure
@terminal /fix         →  Fix the error shown in terminal
```

5.4 Running Terminal Commands from Chat

Copilot Chat can execute terminal commands directly. When Copilot suggests a command like `npm install jest`, it will ask for confirmation before running.

Approval Options:

Option	Behavior
Allow once	Run this command now, ask again next time
Always allow	Auto-approve this command in the future
Skip	Don't run this command

Auto-Approve Configuration (in `settings.json`):

```
{
  "github.copilot.chat.terminal.autoApprove": {
    "npm init": true,
    "npm install": true
  }
}
```

> **Warning:** Only auto-approve **safe, non-destructive commands**. Never auto-approve commands like `rm`, `drop`, or `reset`.

5.5 Exporting Chat Sessions

You can export your entire Copilot Chat conversation as a JSON file for debugging or documentation:

1. Open the Command Palette: `Cmd+Shift+P` (Mac) / `Ctrl+Shift+P` (Windows)
2. Search for **"Chat: Export"**
3. Save the JSON file

The exported JSON reveals:

- Every prompt and response
- Tool calls made by Copilot
- Sub-agent invocations and their prompts
- Context passed at each step

This is extremely useful for **debugging** why Copilot behaved a certain way.

---

6. Context Window & Token Management

6.1 What Is a Context Window?

The **context window** is the maximum amount of information an AI model can "see" and process at one time during a conversation.

```
┌──────────────────────────────────────┐
│            CONTEXT WINDOW            │
│                                      │
│   ┌──────────────────────────────┐   │
│   │  Instruction files           │   │
│   ├──────────────────────────────┤   │
│   │  Your prompt                 │   │
│   ├──────────────────────────────┤   │
│   │  Copilot's response          │   │
│   ├──────────────────────────────┤   │
│   │  Previous messages           │   │
│   ├──────────────────────────────┤   │
│   │  Attached files / code       │   │
│   └──────────────────────────────┘   │
│                                      │
│  When full → oldest messages removed │
└──────────────────────────────────────┘
```

Everything inside the context window is measured in **tokens**.

What Are Tokens?

Tokens are the units of text that AI models process. A token is roughly:

- ~4 characters in English
- ~¾ of a word

For example, the sentence "GitHub Copilot is great" is approximately 5 tokens.

If a model has a **200K token context window**, it can process about 200,000 tokens of combined input and output at once. That's roughly 150,000 words — enough for a small novel, but it fills up faster than you'd think when including code files, chat history, and instruction files.

What Happens When the Context Window Is Full?

When the context window reaches capacity:

1. The **oldest messages** are silently removed
2. Copilot **forgets earlier instructions** and context
3. Responses may become **inconsistent** or **miss requirements**

This is why context management is critical for productive Copilot usage.

6.2 Context Pollution

Context pollution occurs when irrelevant information from earlier in a conversation interferes with later prompts.

Example of Context Pollution

```
You: Build a login form with email validation
Copilot: [generates login form code]

You: Now create a product listing page
Copilot: [generates product page... but accidentally includes
          login-related validation logic because the login context
          is still in the window]
```

The login form context "polluted" the product listing task.

6.3 Best Practices for Context Management

Start a New Chat for Each Task

This is the single most important practice. Start a **fresh chat** for each distinct task:

- Adding a new feature
- Fixing a bug
- Creating a component
- Writing tests

A new chat = a clean context = better results.

Use Small, Focused Prompts

Break complex tasks into small steps instead of asking for everything at once:

❌ **Bad — one massive prompt:**

```
Build authentication, dashboard UI, and API integration.
```

✅ **Good — step by step:**

```
Step 1: Create the login form component
Step 2: Add form validation with Zod
Step 3: Connect the form to the auth API
```

Benefits:

- Copilot generates more accurate code
- Easier for you to review each step
- Consumes less context window space
- Fewer errors and inconsistencies

6.4 Viewing Context Usage in VS Code

You can monitor how much of the context window is being used:

1. Look for the **pie chart icon** at the bottom of the Copilot Chat panel
2. Click or hover over it to see:
    - **Percentage used** (e.g., "Context usage: 17%")
    - **Breakdown** of what's consuming context (instruction files, prompts, responses)

If context usage is getting high, it's time to start a new chat.

---

7. Agent Instructions & Configuration Files

Agent instructions are **Markdown files containing rules and coding standards** that influence how GitHub Copilot generates code. They are the key to getting **consistent, project-specific** output from Copilot.

Why Instructions Matter

Without instructions, Copilot generates code based on **general best practices** — which may not match your project's:

- Framework conventions (Next.js App Router vs Pages Router)
- State management approach (Redux vs Zustand vs Context)
- Styling system (Tailwind vs CSS Modules vs styled-components)
- Data fetching patterns (Server Components vs client-side fetching)
- Error handling conventions

With instructions, Copilot **adheres to your specific patterns**, producing code that fits naturally into your project.

```
Without Instructions:
  Prompt → Copilot → Generic code (may not fit your project)

With Instructions:
  Prompt + Instructions → Copilot → Project-specific code ✓
```

7.1 copilot-instructions.md

Location: `.github/copilot-instructions.md`

This is the **primary instruction file** for GitHub Copilot. When this file exists, Copilot **automatically attaches it as context to every chat** — you don't need to reference it manually.

What to Include

- Project overview and architecture

- Tech stack and framework conventions
- Coding standards and naming conventions
- File structure guidelines
- Component patterns
- API and data fetching patterns

Example

```
# Project Instructions

## Tech Stack

- Next.js 14 with App Router
- TypeScript (strict mode)
- Tailwind CSS for styling
- Drizzle ORM for database access
- Zod for input validation
- shadcn/ui component library

## Coding Standards

- Use functional components only (no class components)
- All components must be in `src/components/`
- API routes go in `src/app/api/`
- Use server actions for mutations
- Never use `any` type — always define proper types

## Error Handling

- Use try/catch blocks in all async functions
- Return structured error responses from API routes
- Log errors to the console in development
```

How to Create It

Method 1: From Copilot Chat settings gear icon → "Chat Instructions" → Create new file

Method 2: Manually create the file:

```
mkdir -p .github
touch .github/copilot-instructions.md
```

7.2 AGENTS.md

Location: `AGENTS.md` (project root)

AGENTS.md is a **general-purpose AI instruction file** that works across **multiple AI coding tools**, not just GitHub Copilot.

Which Tools Read AGENTS.md?

Tool	Reads AGENTS.md?
GitHub Copilot	✅ Yes
Cursor IDE	✅ Yes
Claude Code	✅ Yes
Windsurf	✅ Yes
Other AI agents	✅ Most do

Example

```
# AI Coding Instructions

## Stack

- Use Next.js App Router
- Use TypeScript for all files
- Use Drizzle ORM for database queries
- Use Neon Postgres as the database
- Use Zod for validation
- Use shadcn/ui components

## Rules

- All API calls must go through `/lib/api`
- React components should use functional style
- CSS must use Tailwind utility classes only
- No inline styles allowed
```

When to Use AGENTS.md vs copilot-instructions.md

Scenario	Use
Team uses only GitHub Copilot	`copilot-instructions.md`
Team uses multiple AI tools	`AGENTS.md`
Want maximum compatibility	Both (Copilot reads both)

7.3 .github/agents/*.md

Location: `.github/agents/frontend.md`, `.github/agents/backend.md`, etc.

For **large projects**, you can split AI instructions into layer-specific files instead of one massive instruction file.

Example Structure

```
.github/
└── agents/
    ├── frontend.md     →  React/UI rules
    ├── backend.md      →  API and server rules
    └── database.md     →  Database and ORM rules
```

Example: `frontend.md`

```
# Frontend Agent Instructions

- Use React functional components
- Style with Tailwind CSS utility classes
- Use shadcn/ui for common UI patterns
- Implement responsive design (mobile-first)
- Use React Hook Form for form handling
- Validate with Zod schemas
```

Why Split Instructions?

❌ **Problem — one huge file:**

```
AGENTS.md  (2000+ lines — too much context, hard to maintain)
```

✅ **Better — modular files:**

```
.github/agents/frontend.md   (focused, small)
.github/agents/backend.md    (focused, small)
.github/agents/database.md   (focused, small)
```

AI reads **smaller, focused instructions** → better code generation → less context wasted.

7.4 .github/instructions/*.instructions.md

Location: `.github/instructions/data-fetching.instructions.md`

This is the **official GitHub Copilot-specific** approach for granular instruction files. These files use **YAML frontmatter** to control when and where they are applied.

File Structure

```
---
description: Read this file to understand how to fetch data in the
project.
applyTo: "**/*.tsx"
---

# Data Fetching Guidelines

- Use server components for data fetching when possible
- Use `fetch()` with Next.js caching options
- Always handle loading and error states
- Use Suspense boundaries for streaming
```

Frontmatter Properties

Property	Required?	Purpose
`description`	**Yes**	Tells Copilot **when** to use these instructions. Write it as guidance for the AI model.
`applyTo`	No	Glob pattern restricting which file types these instructions apply to. Omit to apply universally.

How `description` Works

The `description` field is critical — Copilot reads this to **decide whether to include the file as context**. Write it as a directive:

✅ Good: `"Read this to understand authentication patterns in the project"` ❌ Bad: `"Authentication docs"`

How `applyTo` Works

```
applyTo: "**/*.tsx"          # Only when editing .tsx files
applyTo: "src/api/**"        # Only when editing files in src/api/
applyTo: "**/*.test.ts"      # Only when editing test files
```

If omitted, the instructions apply to **all files**.

Managing Instructions in VS Code

- Click the **Settings cog** in Copilot Chat → **Chat Instructions**
- View all detected instruction files
- Create new instruction files from this interface
- Instructions are automatically discovered and applied based on their descriptions

---

7.5 Comparison Table

File	Location	Scope	Used By	Auto-loaded?
`copilot-instructions.md`	`.github/`	Copilot only	GitHub Copilot	✅ Always
`AGENTS.md`	Project root	All AI tools	Copilot, Cursor, Claude, etc.	✅ Always
`.github/agents/*.md`	`.github/agents/`	Module-specific	AI agents	✅ Discovered
`*.instructions.md`	`.github/instructions/`	Conditional	GitHub Copilot	🔄 Based on description & applyTo

Visual Overview

```
project/
│
├── AGENTS.md                                  →  Global AI rules (all tools)
│
└── .github/
    ├── copilot-instructions.md                →  Copilot-specific rules
    ├── agents/
    │   ├── frontend.md                        →  Frontend rules
    │   ├── backend.md                         →  Backend rules
    │   └── database.md                        →  Database rules
    └── instructions/
        ├── data-fetching.instructions.md      →  Conditional rules
        └── testing.instructions.md            →  Conditional rules
```

---

8. Custom Agents

Custom Agents let you create **specialized AI assistants** with restricted tool access and pre-configured behavior. Unlike Agent Mode (which has access to all tools), custom agents only have access to the tools you specify.

Why Use Custom Agents?

In **Agent Mode**, all tools are enabled — including every MCP server you've configured. This means:

- More tool descriptions loaded into context → uses more context window
- The model evaluates more tools per prompt → slower and potentially less accurate
- Irrelevant tools may interfere with responses

Custom agents solve this by exposing only **relevant tools** for a specific task.

Creating a Custom Agent

Create agent definition files in `.github/agents/`:

```
---
name: instructions-generator
description: Generates instruction files for project documentation
tools:
  - read
  - edit
  - search
---

# Instructions Generator Agent

You are a specialized agent that creates `.instructions.md` files
for the project's `.github/instructions/` directory.

## Behavior

- Generate concise, focused instruction files
- Include proper YAML frontmatter with description and applyTo
- Follow existing instruction file patterns in the project
- Ask the user for clarification if information is insufficient
```

Custom Agent vs Agent Mode

Feature	Agent Mode	Custom Agent
Tools available	All (including all MCP)	Only specified tools
Context usage	Higher (all tool descriptions loaded)	Lower (fewer tools)
Precision	General purpose	Task-specific
Configuration	None needed	Requires definition file

Example Use Case

Create an `instructions-generator` agent whose sole purpose is generating well-structured instruction files for your project. When invoked, it:

1. Reads the current project structure
2. Generates a new `.instructions.md` file
3. Creates proper YAML frontmatter
4. Places the file in `.github/instructions/`

---

9. Custom Prompts (Reusable Slash Commands)

Custom Prompts are **reusable prompt templates** that appear as slash commands in Copilot Chat. They let you standardize common workflows so any team member can execute them with a single command.

Creating a Custom Prompt

1. Create a `.github/prompts/` directory
2. Add a `.prompt.md` file (e.g., `create-instructions.prompt.md`)

Custom Prompt Structure

```
---
agent: instructions-generator
---

Take the information below and generate a [name].instructions.md file
for it in the /.github/instructions directory.

Generate an appropriate name for the [name] placeholder based on the
generated content.

Make sure the instructions are concise and not too long.

The .md file should have front matter with a description property that
informs Copilot of when to use this set of instructions.

If no information is provided below, prompt the user to give the
necessary details about the layer of architecture or coding standards
to document.
```

How to Use Custom Prompts

1. Type `/` in Copilot Chat to see available slash commands
2. Select your custom prompt (e.g., `/create-instructions`)
3. Add your details below the prompt template
4. Submit — Copilot switches to the configured agent and executes

When to Create Custom Prompts

- **Repeatable workflows** (generating components, writing tests, creating docs)
- **Team standardization** (everyone follows the same prompt format)
- **Complex multi-step tasks** (the prompt encodes the steps so you don't forget)

---

10. MCP — Model Context Protocol

What Is MCP?

Model Context Protocol (MCP) is a standard that allows AI models to interact with **external tools and services**. It exposes a collection of tools from third-party services (databases, APIs, cloud platforms) to

Copilot Chat, enabling it to perform actions beyond just reading and writing code.

How MCP Works

```
Your Prompt
    ↓
Copilot analyzes the prompt
    ↓
Determines which MCP tools to call
    ↓
Calls external service via MCP
    ↓
Returns result to you
```

For example, if you've connected a **Neon database** via MCP, you can say:

```
Show me all users who signed up last week
```

Copilot will:

1. Identify that the Neon MCP tools are needed
2. Call the appropriate MCP tool to query the database
3. Return the results in your chat

Setting Up MCP in VS Code

1. Open Settings: `Cmd+Shift+P` → search "User Settings"
2. Search for **"Chat Gallery MCP"** → enable it
3. Go to **Extensions** → search with `@tag:mcp`
4. Install the desired MCP server (e.g., Neon, GitHub, etc.)
5. Authenticate the MCP server when prompted

Managing MCP Tools

- Click the **Tools icon** (wrench) in the Copilot Chat window
- View all available tools (built-in + MCP)
- **Enable/disable** individual MCP servers or tools per chat
- Built-in tools include: code editing, terminal commands, file operations

Key Concepts

Concept	Detail
Tool discovery	MCP front-loads all tool descriptions into context at chat start
Approval required	Each MCP tool call requires user approval ("Allow" button)
Context cost	MCP tools consume more context window than Agent Skills

Concept	Detail
Tool selection	Copilot automatically selects tools based on your prompt wording

MCP vs Agent Skills

Feature	MCP	Agent Skills
Context loading	**Front-loads** all tools at start	**Progressive** — loads only when needed
Context window impact	High	Low
Purpose	External service integration	Custom instructions & scripts
Discovery	Always visible	Model reads description first

> **Tip:** When possible, prefer Agent Skills over MCP to conserve context window space.

---

11. Agent Skills

What Are Agent Skills?

Agent Skills are **folders of instructions, scripts, and resources** that Copilot agents can discover and use on demand. Think of them as **custom tools** you create for your AI agents — but unlike MCP, they use **progressive disclosure** to minimize context window usage.

How Progressive Disclosure Works

```
1. Copilot loads only the skill's short DESCRIPTION into context
2. When a prompt matches the skill's purpose, Copilot reads the full
instructions
3. Scripts and references are loaded only if needed
```

This means idle skills cost almost nothing in context, unlike MCP tools which front-load everything.

Anatomy of an Agent Skill

```
.agents/
└── skills/
    └── my-skill/
        ├── SKILL.md          ← Required: name, description, license
        ├── scripts/           ← Optional: executable scripts
        │   └── run.py
        └── references/        ← Optional: additional documentation
            └── guide.md
```

SKILL.md Structure

```
---
name: monthly-links-chart
description: Generates a chart image showing monthly link statistics
license: MIT
---

# Monthly Links Chart

Instructions for generating the monthly links chart...
```

Installing Skills

Skills can be installed from the community repository at **skills.sh**:

1. Find a skill on the site
2. Copy the install command
3. Choose the scope:
   - **Project scope** — committed to source control, shared with team
   - **Global scope** — available across all projects, not in source control
4. Choose the install method:
   - **Copy** — if using one AI coding agent
   - **Symlink** — if switching between agents (e.g., Copilot + Claude Code)

> **Security Warning:** Agent Skills can execute scripts on your local machine. Only install skills from **trusted providers** (e.g., Vercel Labs, Anthropic).

The Skill Creator Skill

A recommended meta-skill from Anthropic that helps you **create new custom skills** via prompting:

1. Install the `skill-creator` skill from skills.sh
2. Use it in chat by typing `/skill-creator` or by naturally prompting (e.g., "create a skill that generates weekly reports")

Invoking Skills

Skills can be invoked in two ways:

1. **Slash command:** `/monthly-links-chart`
2. **Natural language:** "Generate the monthly links chart" — Copilot matches the prompt to the skill's description automatically

---

12. SubAgents

What Are SubAgents?

SubAgents allow Copilot Chat to **delegate tasks to isolated child agents**, each with their own separate context window. This is a powerful technique for handling complex, multi-part tasks without running into

context pollution or context window limits.

The Problem SubAgents Solve

```
Without SubAgents:
  Task 1 context  ──┐
  Task 2 context  ──┼──→ Single context window → OVERFLOW / POLLUTION
  Task 3 context  ──┘

With SubAgents:
  Main Agent ──┬──→ SubAgent 1 (own context) → Task 1 ✓
               ├──→ SubAgent 2 (own context) → Task 2 ✓
               └──→ SubAgent 3 (own context) → Task 3 ✓
```

How SubAgents Work

1. The **main chat agent** identifies tasks that can be delegated
2. It fires off separate **sub-agents**, each with a focused prompt
3. Each sub-agent works in **isolation** — no shared context
4. Sub-agents **report back** with a simple success/failure result
5. The main agent continues without the sub-agents' full context

How to Use SubAgents

Include instructions in your prompt that tell Copilot to use sub-agents:

```
Analyze the codebase for security issues.

Next, ask the user which issues they want to fix by either replying
"all" or a comma-separated list of IDs.

After their reply, run a separate sub-agent (#run-sub-agent) to fix
each issue that the user has specified.

Each sub-agent should report back with a simple
`sub-agent-success: true/false`.
```

- `#run-sub-agent` is a **built-in tool** in Copilot Chat (visible in the Agent tools list)
- Copilot may intelligently **group related issues** into a single sub-agent

What Sub-Agents Receive

Each sub-agent gets:

Field	Content
Description	What the sub-agent should do (e.g., "Fix issues 1 and 7 — open redirect")

Field	Content
Prompt	Specific issue details and context
Task	Step-by-step instructions (read file, add validation, test, etc.)

Verifying Sub-Agent Usage

1. Export the chat as JSON (`Cmd+Shift+P` → "Chat: Export")
2. Search the JSON for `run-sub-agent` tool calls
3. Inspect the description, prompt, and task sent to each sub-agent

---

13. Hooks

What Are Hooks?

Hooks allow you to **execute custom shell commands** at key points during Copilot agent execution. They're essentially lifecycle callbacks that automate tasks like formatting code, running linters, or executing tests after every Copilot action.

Available Hook Types

Hook	When It Fires
`SessionStart`	When a chat session begins
`SessionEnd`	When a chat session ends
`PostPrompt`	After a user submits a prompt (before response)
`PreToolUse`	Before a tool is used (e.g., before writing code)
`PostToolUse`	After a tool is used (e.g., after writing code)
`ErrorOccurred`	When an error occurs during execution

Setting Up Hooks

1. Create a `.github/hooks/` directory in your project root
2. Create a JSON file inside it (any name, e.g., `format.json`)
3. Define the hooks you want

Hook JSON Structure

```
{
  "PostToolUse": [
    {
      "type": "command",
      "bash": "npx prettier --write ."
    }
```

```
  ]
}
```

Property	Purpose
`type`	Always `"command"`
`bash`	The shell command to run (or path to a bash script)
`powershell`	Optional equivalent for Windows environments

The hook accepts an **array** — you can chain multiple commands:

```
{
  "PostToolUse": [
    {
      "type": "command",
      "bash": "npx prettier --write ."
    },
    {
      "type": "command",
      "bash": "npx eslint --fix ."
    }
  ]
}
```

Important Considerations

- The JSON file can be **any name** as long as it's inside `.github/hooks/`
- `PostToolUse` fires for **every** tool use — including file reads, not just writes
- To avoid unnecessary runs, **scope hooks to specific tools** (e.g., only run the formatter after file edit/create, not after file read)
- Hooks work in both **Copilot CLI** and **Copilot Chat**

Common Use Case: Auto-Formatting

The most common hook: run Prettier (or your preferred formatter) after every code generation, so Copilot's output always matches your project's formatting rules.

```
{
  "PostToolUse": [
    {
      "type": "command",
      "bash": "npx prettier --write ."
    }
  ]
}
```

Running Commands in Copilot CLI

In the Copilot CLI, you can run shell commands directly by prefixing with `!`:

```
!npx prettier --write .
```

Use the `/diff` command to see all files changed from a particular prompt. Navigate between files with arrow keys.

---

14. AI-First Project Structure

When building a project with AI tools, the file structure you choose matters. A well-organized project with proper AI instruction files results in dramatically better code generation.

Recommended Structure

```
project/
│
├── AGENTS.md                                  ← Global AI rules (all
tools)
│
├── docs/                                      ← Modular documentation
│   ├── architecture.md
│   ├── coding-standards.md
│   ├── auth.md
│   ├── database.md
│   ├── api.md
│   ├── ui.md
│   ├── testing.md
│   └── deployment.md
│
├── .github/
│   ├── copilot-instructions.md                ← Copilot-specific rules
│   ├── hooks/
│   │   └── format.json                        ← Auto-format after code gen
│   ├── agents/
│   │   ├── frontend-agent.md                  ← Frontend custom agent
│   │   ├── backend-agent.md                   ← Backend custom agent
│   │   └── instructions-generator.md          ← Doc generator agent
│   ├── prompts/
│   │   └── create-instructions.prompt.md      ← Reusable prompt template
│   └── instructions/
│       ├── data-fetching.instructions.md      ← Conditional instructions
│       ├── testing.instructions.md
│       └── auth.instructions.md
│
├── .agents/
│   └── skills/                                ← Custom agent skills
│       └── my-skill/
```

```
│           ├── SKILL.md
│           └── scripts/
│
├── src/
│   ├── app/
│   ├── components/
│   ├── lib/
│   ├── services/
│   └── types/
│
├── package.json
└── README.md
```

The Documentation System Workflow

Instead of letting AI randomly generate documentation, build a **structured system**:

```
Step 1: Define global rules          →  AGENTS.md / copilot-
instructions.md
Step 2: Create layer documentation   →  docs/auth.md, docs/ui.md, etc.
Step 3: Reference docs in AGENTS.md  →  "See docs/auth.md for auth rules"
Step 4: Use custom agents to generate docs cleanly
Step 5: Use custom prompts to standardize the process
```

Flow:

```
Developer prompt
       ↓
Custom Agent (instructions-generator)
       ↓
Creates docs/*.md with clean, structured content
       ↓
Updates AGENTS.md to reference the new docs
       ↓
Copilot reads all instructions → Generates better, project-specific code
```

Why This Structure Works

1. **Controlled AI behavior** — AI follows structured rules, not random generation
2. **Modular instructions** — Small, focused files instead of one massive file
3. **Reusable generation** — Easily generate new docs (e.g., `docs/payment.md`)
4. **Scalable** — Works for large projects with many layers (frontend, backend, auth, database, API, testing)

---

15. Best Practices & Tips

Context Management

1. **Start a new chat for each task** — prevents context pollution and ensures a clean context window
2. **Break prompts into small steps** — easier for Copilot, easier for you to review
3. **Monitor context usage** — use the pie chart icon in VS Code to check consumption
4. **Use Plan Mode first** — outline the approach, then switch to Agent Mode to execute

Instructions & Configuration

5. **Create instruction files for every architectural layer** — auth, data fetching, mutations, UI, routing, testing
6. **Use `description` frontmatter wisely** — it determines whether Copilot includes the file as context
7. **Use `applyTo` patterns** — restrict instructions to relevant file types
8. **Keep instructions concise** — avoid putting everything in one giant file

Agents & Tools

9. **Use custom agents** to narrow tool access for specific workflows
10. **Prefer Agent Skills over MCP** when possible — they use progressive disclosure and consume less context
11. **Use sub-agents** for multi-issue fixes to keep context clean and isolated
12. **Scope hooks to specific tools** — only run formatters on file edit/create, not on file read

Code Generation

13. **Use comments to guide inline suggestions** — descriptive comments = better autocomplete
14. **Use `/tests` to generate test coverage quickly** — select code first for best results
15. **Create custom prompts** for repeatable workflows — standardize across the team
16. **Export chats as JSON** to debug and understand Copilot's behavior

Ask Mode for Exploration

17. **Use Ask Mode** for exploratory questions without modifying code
18. **Use `@workspace /explain`** to understand project-level code
19. **Use `@terminal`** to debug terminal errors through Copilot

---

16. Quick Reference Cheat Sheet

File Locations

File	Location	Purpose
`AGENTS.md`	Root	Global AI rules (all tools)
`copilot-instructions.md`	`.github/`	Copilot-specific rules
`*.md`	`.github/agents/`	Custom agent definitions
`*.instructions.md`	`.github/instructions/`	Conditional instructions

File	Location	Purpose
`*.prompt.md`	`.github/prompts/`	Custom slash commands
`*.json`	`.github/hooks/`	Lifecycle hooks
`SKILL.md`	`.agents/skills/*/`	Agent skill definitions

Keyboard Shortcuts (VS Code)

Action	Mac	Windows
Accept suggestion	`Tab`	`Tab`
Accept word	`Cmd+→`	`Ctrl+→`
Next suggestion	`Alt+]`	`Alt+]`
Previous suggestion	`Alt+[`	`Alt+[`
Dismiss suggestion	`Esc`	`Esc`
Open Command Palette	`Cmd+Shift+P`	`Ctrl+Shift+P`
Export Chat	`Cmd+Shift+P` → "Chat: Export"	`Ctrl+Shift+P` → "Chat: Export"

Chat Modes

Mode	Shortcut	Modifies Code?
Ask	Select "Ask" in chat	No
Plan	Select "Plan" in chat	No
Agent	Select "Agent" in chat	Yes

Essential Slash Commands

Command	Action
`/setup-tests`	Configure testing framework
`/tests`	Generate unit tests
`/explain`	Explain code
`/fix`	Fix a problem
`/doc`	Generate documentation

Chat Participants

Participant	Purpose
`@workspace`	Entire project context

Participant	Purpose
`@terminal`	Terminal output context
`@editor`	Current file context

> **One-Line Summary:** Master GitHub Copilot by combining **inline suggestions** for fast coding, **instruction files** for consistent output, **custom agents** for specialized workflows, **MCP** for external integrations, and **sub-agents** for complex multi-step tasks — all while managing your **context window** carefully.

This guide was compiled from course transcripts and organized as a structured developer handbook for GitHub Copilot.